

Project management platform

Frontend Assignment
Pinzaru Madalin



Project description

Web application built with React and TypeScript for managing Projects, Organizations, Users and viewing statistics

Key Features

CRUD Operations

Form validation and user feedback

Pagination for larger data sets

Internalization support

Unit tests

Tech Stack

React

Typescript

Material-UI

Tanstack React Query

React-hook-form

React-i18n-next

Vite

Vitest





Architecture and project structure

Frontend: The application is a React SPA built with Typescript, that manages all the logic, routing and UI rendering client-side

Backend: All data operations (CRUD) are performed via HTTP requests to a remote API

State & data management: Uses React Query for efficient data fetching, caching, and synchronization with the API

UI & UX: Material-UI provides a consistent, responsive design. Internalization and form validation are integrated for accessibility and usability

Project structure: Organized into components, pages, hooks etc for maintainability and scalability

```
src
├── components
│   ├── common
│   ├── Exercise
│   ├── Organizations
│   └── Projects
│       ├── CreateProjectPage.tsx
│       ├── EditProjectPage.tsx
│       ├── ProjectForm.tsx
│       ├── Projects.test.tsx
│       └── Projects.tsx
│   ├── Statistics
│   └── Users
├── hooks
├── styles
├── translations
└── utils
```



Core components

Menu: Provides top level navigation between the pages

Pages: Display, create and edit entities, each page uses forms and tables, supporting CRUD operations and pagination

Forms : Used for both creation and editing, implements real-time validation and error handling

Statistics: Displays aggregated data helping users visualise the data they need

Each component is designed for reusability and maintainability using a good structure to enable scalability.



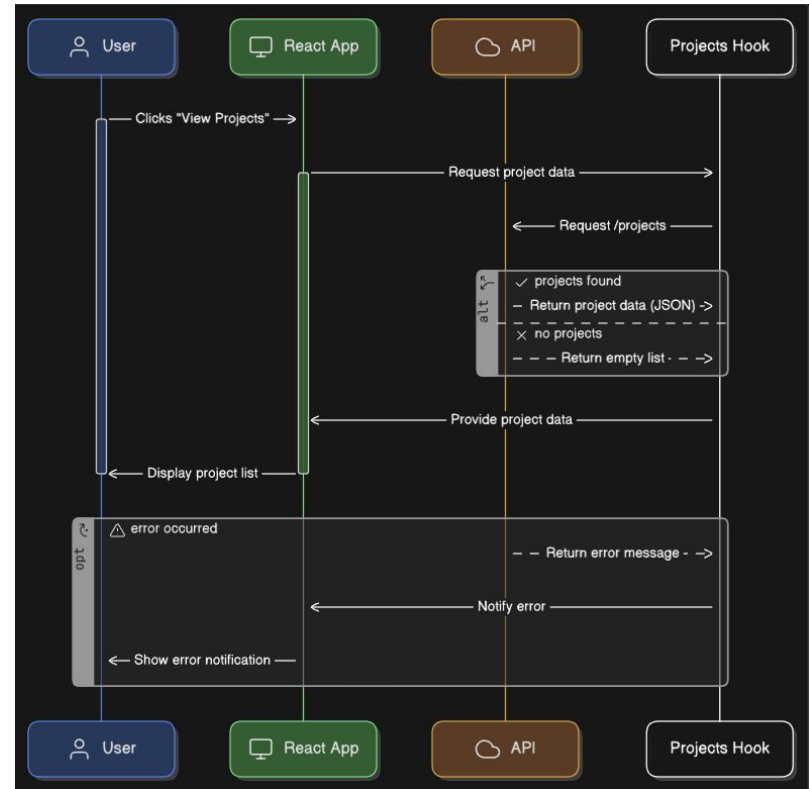
Technologies

React-hook-form: Manages form state and validation efficiently with minimal re-renders, integrates seamlessly with MUI leading to a smooth user experience and reduces overall boilerplate code, making forms easier to maintain and extend

Tanstack React Query: Handles data fetching, caching and synchronization with the API. Automatically updates the UI when data changes, improving responsiveness (e.g. after creating or editing projects) without manual refreshes. Additionally, it simplifies error handling and loading states making it more robust

Detailed API interaction flow for /projects

1. The user clicks "Projects" in the menu
2. The app navigates to the Projects page
3. The project component uses a custom hook to fetch project data
4. The custom hook sends a GET Request to the API
5. The API responds with the projects data
6. The custom hook receives the data and passes it to the component
7. The app displays the projects list to the user



Summary

- Built a modern application to manage projects, organizations and users
- Developed with reusable components, efficient forms and reliable API integration
- Achieved a scalable, maintainable, and user-friendly solution